SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

ASSESSMENT TOOL 2 – SCENARIO BASED ASSIGNMENT

Course: Artificial Intelligence

Course Code	CSA1716	**Course Title**	Artificial Intelligence
CO Assessed	CO2	**Assessment**	Scenario Based Assignment
Weightage	20% of CIA	**Duration / Marks**	60 minutes

CO2 Statement: Experiment search and game-solving techniques such as Greedy Search, A*, CSP, Minimax, and Alpha-Beta pruning with heuristic functions for solving complex AI problems efficiently. (BL3)

Student Name	Rohith
Reg. No.	191126007
Section / Programme	CSE AI
Date	21.10.26

Code of Conduct

I, Rohith (Reg. No. 191126007), certify that this submission is my work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules may result in disciplinary action.

Signature of the Student: _____

Instructions

• Read each scenario carefully before answering (Off-Class Assessment).

• Answer all five scenario-based questions.

• Identify the key issues in each scenario, apply the relevant AI technique, and justify each decision with clear reasoning.

• The questions cover Informed Search (Greedy and A*), Local Search and Optimization, Online Search, Constraint Satisfaction Problems, and Adversarial Search (Minimax, Alpha-Beta pruning, and evaluation functions).

• Marks are awarded for understanding the scenario, identifying key issues, applying concepts, decision making, and clarity of response.

Assignment Topic Map

Section / Topic	Focus	Question Nos.
1. Informed Search Strategies	Greedy best-first search, A* search, heuristic functions	Q1
2. Local Search and Online Search	Hill climbing, simulated annealing, genetic algorithms, online search agent	Q2, Q3
3. Constraint Satisfaction Problem	Variables, domains, constraints, backtracking, constraint propagation and forward checking	Q4
4. Game Playing: Adversarial Search	Minimax, alpha-beta pruning, evaluation functions	Q5

Rubrics for Scenario Based Assignment

Criteria	Weight	Level 4 – Exemplary (100%)	Level 3 – Proficient (75%)	Level 2 – Developing (50%)	Level 1 – Beginning (25%)
Understanding of Scenario	20%	Clear and complete understanding of the scenario and context.	Good understanding with minor gaps.	Partial understanding; misses key aspects.	Misinterprets or fails to understand the scenario.
Identification of Key Issues	20%	Accurately identifies all relevant issues and factors.	Identifies most key issues with minor omissions.	Identifies limited issues; includes irrelevant points.	Unable to identify key issues.
Application of Concepts	25%	Applies appropriate concepts effectively.	Applies concepts with minor errors.	Limited or partially correct application.	Unable to apply relevant concepts.
Decision Making	20%	Logical decisions with strong rationale.	Reasonable decisions with some justification.	Weak decisions with limited justification.	No clear decisions or rationale.
Clarity of Response	15%	Clear, structured, logical response.	Generally clear with minor issues.	Somewhat unclear or poorly structured.	Disorganized and difficult to understand.

Faculty In-charge: _____ Course Coordinator: _____ Program Director: _____

Signature: _____ Date: _____

Annexure A – Scenario Based Assignment

Question 1: Emergency Medicine Delivery Drones in a Flood Zone

Topic: Informed search strategies (Greedy best-first search, A*, heuristic functions) | **12 minutes**

A government deploys AI-powered drones to deliver emergency medicines in a flood-affected region. The environment is highly dynamic: roads may suddenly become inaccessible, and weather conditions affect travel cost. The drone has access to a straight-line-distance heuristic to the destination, partial real-time updates about blocked paths, and variable movement costs depending on terrain and weather. Due to urgency, the drone must quickly decide routes with minimum delay and risk.

Tasks: (a) Explain Greedy Best-First Search and its strengths/weaknesses under uncertainty. (b) Explain A* and how it balances cost and heuristic. (c) Analyze heuristic accuracy. (d) Discuss dynamic blocked paths. (e) Recommend the most suitable algorithm.

Answer

(a) Greedy Best-First Search

Greedy Best-First Search selects the frontier node that appears closest to the destination according to the heuristic h(n). In this scenario, the drone would prioritize the route with the smallest estimated straight-line distance to the medicine delivery point. Its main strength is speed: it can often find a promising route quickly, which is useful during emergencies. However, it ignores the cost already spent to reach a node. A route that looks geographically short may contain strong winds, flooded areas, or unsafe terrain, so Greedy search may produce a costly or risky route. Partial or outdated path information can also mislead it.

(b) A* Search

A* evaluates each node using **f(n) = g(n) + h(n)**, where g(n) is the actual cost accumulated from the start and h(n) is the estimated remaining cost to the destination. The drone can represent delay, energy use, and risk in g(n), while h(n) estimates the remaining travel effort. Unlike Greedy search, A* considers both the route already travelled and the estimated distance left. If the heuristic never overestimates the true minimum remaining cost and the cost model is appropriate, A* can find an optimal path for the modeled problem.

(c) Impact of heuristic accuracy

A useful heuristic guides search toward the goal without excessive exploration. Straight-line distance is generally a lower-bound estimate of distance, but it may not accurately predict travel time or risk when winds, water, or obstacles affect the route. A weak heuristic makes A* explore more nodes; an overestimating heuristic can cause A* to lose its optimality guarantee. Greedy search depends heavily on h(n), because it uses the heuristic alone to rank frontier nodes. Therefore, stale or misleading estimates can send it in the wrong direction.

(d) Dynamic blocked paths

When a path becomes blocked, the drone must mark the affected edge or area as unavailable, update relevant movement costs, and replan from its current position. Repeatedly restarting a full search can waste time, so an incremental replanning method or frequent limited re-evaluation can be useful. Sensor reports should be checked for freshness, and the route planner should preserve safety margins when information is uncertain. The drone must not continue along a route merely because it was optimal before the environment changed.

(e) Recommendation

A* is the better default when safety and total travel cost matter, because it considers both accumulated cost and estimated remaining cost. It should use a cost function that penalizes unsafe conditions and a heuristic that remains admissible for the objective being optimized. Since the environment changes, A* should be combined with real-time obstacle updates and replanning. If an immediate emergency requires a very fast provisional decision, Greedy search may provide a quick candidate, but the route should still be checked for risk and updated as new information arrives.

Question 2: Smart City Traffic Signal Optimization

Topic: Local search algorithms and optimization problems (hill climbing, simulated annealing, genetic algorithms) | **12 minutes**

A smart city implements an AI system to optimize traffic signal timings across hundreds of intersections. The objectives are to minimize total waiting time, fuel consumption, and traffic congestion. The system must continuously adjust signals based on traffic flow. The search space is extremely large, traffic patterns change dynamically, and the system may get stuck in locally optimal solutions.

Tasks: (a) Formulate the optimization problem and explain why traditional search is inefficient. (b) Explain Hill Climbing and its limitations. (c) Explain local maxima, plateaus, and ridges. (d) Explain how Simulated Annealing or Genetic Algorithms can overcome limitations. (e) Recommend an approach.

Answer

(a) Formulation as an optimization problem

A candidate solution is a set of signal timings and phase sequences for the intersections. Decision variables may include green-light duration, cycle length, phase order, and coordination offsets between nearby signals. The objective function can combine total vehicle waiting time, fuel use, and congestion, for example **J = w■W + w■F + w■C**, where W is waiting time, F is fuel consumption, C is congestion, and the weights w■, w■, and w■ express their relative importance. The system seeks to minimize J while obeying safety rules, minimum and maximum green times, pedestrian crossing times, and intersection capacity constraints. Traditional exhaustive search is inefficient because the number of possible combinations grows rapidly with the number of intersections and timing choices.

(b) Hill Climbing

Hill Climbing starts with a feasible timing plan, evaluates neighboring plans, and moves to a neighbor that improves the objective. For example, it may extend a green phase on a congested road or adjust offsets to reduce stopping. It is simple and can improve a plan quickly, but it usually accepts only improvements. It can stop at a local optimum, depend strongly on the starting plan, and fail to discover a better solution that requires temporarily accepting a worse score.

(c) Local maxima, plateaus, and ridges

A **local optimum** occurs when nearby timing changes do not improve the score, even though a better city-wide plan may exist. A **plateau** is a region where many neighboring plans have almost equal scores, so the search has little guidance. A **ridge** occurs when improvement requires a coordinated sequence of changes across several variables; changing only one intersection may not help. In a city, one signal's timing can affect queues at adjacent intersections, so isolated local adjustments may fail to improve the whole network.

(d) Simulated Annealing and Genetic Algorithms

Simulated Annealing sometimes accepts a worse solution with a probability controlled by a temperature parameter. Early in the search, this helps it escape local optima; as the temperature decreases, the algorithm becomes more selective and settles toward a good solution. A Genetic Algorithm maintains a population of candidate timing plans, selects better-performing plans, and creates new candidates through crossover and mutation. This explores multiple regions of the search space at once. Both methods need careful parameter tuning and evaluation, and neither guarantees a globally optimal answer in a practical time.

(e) Recommendation

A Genetic Algorithm is a suitable choice for large networks because it can explore many candidate plans and optimize interactions among intersections. For real-time operation, use a rolling optimization window, seed the population with the current feasible plan, and evaluate candidates using recent traffic data. A practical hybrid can use a Genetic Algorithm to find strong coordinated plans and a fast local improvement step to refine them. All outputs must be checked against safety constraints before deployment, with fallback timings available if sensor data or computation fails.

Question 3: Autonomous Rover Exploring Mars

Topic: Online search agent | **12 minutes**

An autonomous rover is deployed on Mars to explore unknown terrain. It must navigate safely to collect samples, avoid hazards such as craters and rocks, operate with limited sensing capability, and make decisions without a complete map. The rover updates its knowledge as it explores, making this a real-time decision-making problem.

Tasks: (a) Explain why an online search agent is required. (b) Analyze partial observability and dynamic-environment challenges. (c) Describe how the rover builds and updates its internal model. (d) Compare online search with uninformed and informed search. (e) Justify the most suitable approach.

Answer

(a) Why online search is required

Offline search assumes the relevant state space and action outcomes are known before planning. The rover has no complete map and discovers terrain as it moves. An online search agent alternates between sensing, choosing an action, moving, and updating its knowledge. It is therefore appropriate when the agent must learn the environment during exploration and make decisions before all information is available.

(b) Partial observability and environmental challenges

The rover's sensors may detect only nearby terrain, and rocks or slopes may hide hazards beyond the visible area. Sensor noise can make the same terrain appear different at different times. Communication delays and limited energy also restrict how much information can be sent to Earth. The rover must distinguish known safe regions, known hazards, and unexplored regions, and it should treat uncertain areas cautiously. Although the terrain may not change as quickly as city traffic, new observations can reveal that an earlier map or assumption was wrong.

(c) Building and updating an internal model

The rover maintains a local map or state representation containing its estimated position, visited cells, traversability, hazards, and scientific targets. At each step it gathers camera, range, and other sensor readings; updates the map; revises confidence in uncertain cells; and recalculates reachable routes. It can record the cost of actions such as driving across a slope or around a crater. After detecting an unexpected obstacle, it marks the path as blocked and replans from its current location. Keeping a record of visited states helps avoid repeated exploration without purpose.

(d) Comparison with uninformed and informed search

Uninformed search, such as breadth-first search, does not use goal-distance estimates and may explore many states that do not help reach the target. Informed search uses a heuristic, such as estimated distance to a sample site, to prioritize promising states. However, a conventional informed search plan may assume a known map. Online search emphasizes acting and learning in an unknown environment. An online rover can still use heuristic guidance, but it must revise the plan whenever new observations change the known state space.

(e) Recommendation

Use an online, heuristic-guided search agent with conservative obstacle avoidance, incremental map updates, and replanning after unexpected observations. The heuristic should favor sample locations while accounting for distance, slope, energy, and hazard risk. The rover should prefer known safe routes when uncertainty is high, preserve an energy reserve, and stop or request support if sensor confidence falls below a safe threshold. This approach balances adaptability and scientific progress with the need to protect the rover.

Question 4: University Exam Timetable Generation

Topic: Constraint Satisfaction Problem (backtracking search, constraint propagation, forward checking) | **12 minutes**

A university is designing an AI system to automatically generate exam timetables. Constraints include no student having overlapping exams, a limited number of exam halls, prerequisite scheduling order for certain subjects, faculty availability, and special accommodations for some students. Complexity increases as the number of courses and students grows.

Tasks: (a) Formulate the problem as a CSP. (b) Explain backtracking search. (c) Explain constraint propagation and forward checking. (d) Analyze real-world challenges and the best scalable strategy.

Answer

(a) CSP formulation: variables, domains, and constraints

A Constraint Satisfaction Problem consists of variables, domains of possible values, and constraints restricting combinations of values. For exam scheduling:
Variables: each course exam, represented by its assigned time slot and room (or by a combined time-room value).
Domains: the available exam slots and rooms that can accommodate the course, after removing unavailable times and unsuitable rooms.
Constraints: (1) exams taken by the same student must not overlap; (2) room capacity and number of available halls must be respected; (3) specified subjects must be scheduled in the required order; (4) faculty or invigilator availability must be respected; and (5) accessibility and other approved accommodations must be provided. A valid timetable is an assignment satisfying every hard constraint. Soft preferences, such as avoiding too many exams on one day, can be optimized after feasibility is achieved.

(b) Backtracking search

Backtracking assigns a value to one variable at a time. The system chooses an exam, assigns a feasible time and room, and checks the constraints. If a later assignment conflicts with an earlier one, it undoes the most recent assignment and tries another value. Variable-ordering heuristics such as choosing the exam with the fewest legal options first can reveal conflicts early. Choosing a value that leaves the greatest number of options for other exams can also reduce dead ends.

(c) Constraint propagation and forward checking

After assigning a time and room to an exam, forward checking removes inconsistent values from the domains of unassigned exams. For example, if two courses share students, assigning one course to a particular slot removes that slot from the other's domain. If any unassigned exam has an empty domain, the system detects the conflict immediately and backtracks rather than continuing with an impossible partial timetable. Constraint propagation repeatedly applies such restrictions, reducing unnecessary search. It does not guarantee a solution, but it can substantially reduce the search space.

(d) Real-world challenges and scalable strategy

Challenges include large enrolments, late course changes, incomplete student-registration data, limited room capacity, accessibility requirements, and staff availability. The system should validate input data, model hard constraints explicitly, and separate hard constraints from soft preferences. A scalable strategy combines backtracking with the most-constrained-variable heuristic, degree heuristic, least-constraining-value ordering, and forward checking. For very large instances, the timetable can be decomposed by periods or groups while preserving cross-group conflict constraints. After any change, the system should repair the affected part of the timetable and re-validate the complete schedule.

Question 5: Strategic Game AI for Real-Time Decision Making

Topic: Game playing: adversarial search (Minimax, Alpha-Beta pruning, evaluation functions) | **12 minutes**

A gaming company is developing an AI opponent for a real-time strategy game. The AI must compete against human players, make decisions within strict time limits, and evaluate complex game states with many possible moves. The decision tree is extremely large, making computation expensive.

Tasks: (a) Explain Minimax. (b) Analyze computational challenges of large game trees. (c) Explain Alpha-Beta pruning. (d) Design an evaluation function. (e) Recommend a balance between optimality and real-time performance.

Answer

(a) Minimax algorithm

Minimax models a competitive game as alternating decisions by two players: MAX, the AI, tries to maximize the utility score, while MIN, the opponent, tries to minimize it. The algorithm expands possible moves to a chosen depth, evaluates terminal or cutoff states, and propagates values upward. At a MAX node it selects the largest child value; at a MIN node it selects the smallest. The AI chooses the move with the best worst-case outcome under the assumed opponent model. This is most directly applicable to deterministic, turn-based, zero-sum games with defined legal actions.

(b) Computational challenges

If the average branching factor is b and the search depth is d, ordinary Minimax may examine on the order of b^d states. In a strategy game, each state can offer many unit, movement, resource, and combat actions, so the tree grows rapidly. Strict response deadlines make exhaustive search impractical. The game may also include simultaneous events, hidden information, or randomness, which require a richer model than basic Minimax. State evaluation itself can be expensive if it simulates many future effects.

(c) Alpha-Beta pruning

Alpha-Beta pruning gives the same Minimax decision while avoiding branches that cannot affect the final choice. Alpha is the best value MAX can guarantee so far; beta is the best value MIN can guarantee so far. At a MAX node, alpha is updated when a better value is found. At a MIN node, beta is updated when a lower value is found. When alpha becomes greater than or equal to beta, the remaining children of that node can be pruned because an ancestor already has a choice at least as good. Good move ordering increases pruning: in favorable cases, the effective search can approach O(b^(d/2)) rather than O(b^d), although worst-case behavior remains exponential.

(d) Example evaluation function

At a depth limit, the AI can score a non-terminal state using a weighted function such as:
Eval(s) = 5M + 3R + 4T + 2P − 5L − 4D
where M is the AI's military strength advantage, R is resource advantage, T is territory/control advantage, P is progress toward the objective, L is losses suffered, and D is danger to critical units or base. The numerical weights are illustrative and must be tuned through testing. Features should be normalized so one factor does not dominate only because it uses a larger numerical scale. The function should reflect the actual victory conditions and avoid rewarding a short-term gain that leads to a strategically poor position.

(e) Recommendation

Use depth-limited Minimax with Alpha-Beta pruning, strong move ordering, and a carefully tested evaluation function. Iterative deepening can search depth 1, then 2, then 3, keeping the best completed move available if the deadline arrives. A transposition table can reuse evaluations of repeated states, while selective extensions can focus extra search on important tactical situations. The AI should always have a legal fallback action if time expires. This approach preserves Minimax's decision logic while reducing wasted computation, making it more practical for real-time play.

— END OF ASSIGNMENT —

Student: Rohith | Reg. No.: 191126007 | Course Code: CSA1716 | Date: 21.10.26